RADAR STOCK

Informe Técnico de Construcción y Banco de Pruebas de Desarrollo

Alcance del Documento: Fases 1 a 3 (Base de Datos, Autenticación y Roles, CRUD de Productos y Stock)

Entorno: Android Nativo (Kotlin/XML) | Backend PHP 8 (PDO) | Base de Datos MySQL (XAMPP)

Fecha: Noviembre 2026 | **Estado:** Pruebas Verificadas con Éxito

---

1. Resumen de Arquitectura y Reglas del Proyecto

Radar Stock es una solución de punto de venta e inventario ágil orientada a almacenes de barrio. El desarrollo sigue una arquitectura desacoplada estricta compuesta por tres capas:

- **Capa Móvil (Android):** Kotlin nativo con ViewBinding y RecyclerView. Sin Jetpack Compose. Utiliza Retrofit con OkHttp y corrutinas para llamadas asíncronas seguras, interceptando peticiones para adjuntar tokens Bearer y controlando la caducidad (código 401). Compilado bajo targetSdk 36 para compatibilidad con la red local del emulador.
- **Capa Backend (API REST en PHP):** Respuestas exclusivas en formato JSON UTF-8 mediante funciones centralizadas. Acceso a datos con PDO mediante sentencias preparadas contra inyecciones SQL. display_errors desactivado registrando incidencias en archivo local de log.
- **Capa de Persistencia (MySQL):** Esquema relacional radar_stock_schema.sql e índices estratégicos para optimizar consultas de inventario, caducidades y roles de usuario.

2. Fase 1 (Prompt 1) — Base de Datos, API Base y Listado en Android

Objetivo: Establecer la infraestructura inicial, asegurar la conexión a la base de datos MySQL, proteger carpetas sensibles en Apache y listar el catálogo inicial de productos en la aplicación Android.

Archivos Construidos / Modificados:

- database/radar_stock_schema.sql: Tablas iniciales (usuarios, productos, ventas, detalle_ventas).
- database/02_datos_prueba.sql: 10 productos de muestra y 1 usuario administrador inicial.
- backend-api/config/config.example.php: Plantilla de conexión fuera de Git.
- backend-api/includes/db.php y respuesta.php: Manejadores de base de datos y JSON.
- backend-api/api/salud.php: Healthcheck de base de datos.
- backend-api/.htaccess: Bloqueo de acceso HTTP 403 directo a config/ e includes/.
- android-app/: Modelos, ProductoAdapter, RetrofitClient y ProductosActivity.

Endpoint / Ruta	Método	Propósito de la Prueba	Resultado Esperado	Estado
api/salud.php	GET	Comprobar conexión MySQL (SELECT 1)	HTTP 200 {'estado':'ok'}	**PASÓ**
api/productos.php	GET	Listado inicial de los 10 productos	HTTP 200 con números tipados	**PASÓ**
config/config.php	GET	Intento de acceso web a credenciales	HTTP 403 Forbidden (.htaccess)	**PASÓ**

Android App	UI	Despliegue de RecyclerView en emulador	Carga de productos y reintento	**PASÓ**

3. Fase 2 (Prompt 2) — Autenticación, Tokens de Sesión y Control de Roles

Objetivo: Implementar autenticación segura basada en tokens con caducidad de 12 horas, restringir endpoints por rol en el servidor, e integrar LoginActivity y MenuActivity dinámico en Android.

Archivos Construidos / Modificados:

- database/03_auth.sql: Columnas token y token_expira con índice, y usuario Vendedor.
- backend-api/includes/auth.php: Funciones autenticar() y requerirRol().
- backend-api/api/login.php: POST con BCRYPT, generación de token con random_bytes(32).
- backend-api/api/usuarios.php: Listado de usuarios exclusivo para Administrador.
- backend-api/api/productos.php: Protección de ruta mediante token obligatorio.
- android-app/: SessionManager (SharedPreferences), AuthInterceptor (OkHttp), LoginActivity con logo y MenuActivity con botones por rol.

Prueba / Endpoint	Rol / Token	Parámetros / Cuerpo	Resultado Esperado	Estado
api/login.php	Público	Contraseña equivocada	HTTP 401 'Correo o contraseña incorrectos'	**PASÓ**
api/login.php	Admin	admin@radarstock.cl / admin123	HTTP 200 con token de 64 caracteres	**PASÓ**
api/login.php	Vendedor	vendedor@radarstock.cl / vendedor123	HTTP 200 con token de 64 caracteres	**PASÓ**
api/productos.php	Sin token	Llamada directa sin Authorization	HTTP 401 'Token no proporcionado'	**PASÓ**
api/usuarios.php	Vendedor	Token Bearer Vendedor	HTTP 403 'Acceso denegado'	**PASÓ**
api/usuarios.php	Admin	Token Bearer Administrador	HTTP 200 (Lista id, nombre, correo, rol)	**PASÓ**
MenuActivity	Admin (UI)	Sesión iniciada con Admin	Muestra 6 botones (incluye Administración)	**PASÓ**
MenuActivity	Vendedor (UI)	Sesión iniciada con Vendedor	Muestra solo 3 botones (Venta/Prod/Alert)	**PASÓ**

4. Fase 3 (Prompt 3) — CRUD de Productos, Categorías y Stock Atómico

Objetivo: Expandir la API a un CRUD completo con sentencias preparadas, filtros de búsqueda, paginación, bloqueo transaccional de stock con FOR UPDATE y validaciones de integridad referencial antes de eliminar.

Archivos Construidos / Modificados:

- backend-api/api/productos.php: Soporte GET (filtros y detalle), POST (creación), PUT (edición sin stock) y DELETE (integridad con detalle_ventas).
- backend-api/api/stock.php: Transacción con bloqueo FOR UPDATE para incremento consistente de stock.

• backend-api/api/categorias.php: Combinación de 6 categorías base con categorías en base de datos, sin duplicados y ordenadas alfabéticamente.

Operación / Endpoint	Rol / Token	Detalle de la Prueba	Resultado Esperado	Estado
GET api/productos.php	Vendedor	?buscar=Coca (búsqueda parcial)	HTTP 200 filtra por coincidencia	**PASÓ**
GET api/productos.php	Vendedor	?id=1 (detalle de producto)	HTTP 200 devuelve un único producto	**PASÓ**
GET api/categorias.php	Vendedor	Listado completo sin duplicados	HTTP 200 con categorías ordenadas	**PASÓ**
POST api/productos.php	Vendedor	Intento de crear producto	HTTP 403 Forbidden (solo Admin)	**PASÓ**
POST api/productos.php	Admin	Nuevo producto válido	HTTP 201 Created con objeto creado	**PASÓ**
POST api/productos.php	Admin	Mismo código de barras	HTTP 409 'Ese código ya existe'	**PASÓ**
PUT api/productos.php	Vendedor	Intento de editar producto	HTTP 403 Forbidden (solo Admin)	**PASÓ**
PUT api/productos.php	Admin	Edición de precio y nombre	HTTP 200 modifica campos excepto stock	**PASÓ**
POST api/stock.php	Vendedor	Intento de sumar stock	HTTP 403 Forbidden (solo Admin)	**PASÓ**
POST api/stock.php	Admin	Suma transaccional (+8 unidades)	HTTP 200 stock_actual actualizado	**PASÓ**
DELETE api/productos.php	Vendedor	Intento de eliminar producto	HTTP 403 Forbidden (solo Admin)	**PASÓ**
DELETE api/productos.php	Admin	Producto con ventas en detalle_ventas	HTTP 409 'Tiene ventas registradas'	**PASÓ**
DELETE api/productos.php	Admin	Producto sin ventas	HTTP 200 'Eliminado exitosamente'	**PASÓ**

5. Banco de Pruebas Automatizadas en PowerShell

A continuación se documenta el script de ejecución directa en PowerShell utilizado para validar la totalidad de las rutas de punta a punta con Invoke-RestMethod:

```
# 1. Obtener tokens
$admin = Invoke-RestMethod -Uri "http://localhost/backend-api/api/login.php" `
    -Method POST -ContentType "application/json" `
    -Body '{"correo":"admin@radarstock.cl","password":"admin123"}'
$headersAdmin = @{ Authorization = "Bearer $($admin.token)" }

$vendedor = Invoke-RestMethod -Uri "http://localhost/backend-api/api/login.php" `
    -Method POST -ContentType "application/json" `
    -Body '{"correo":"vendedor@radarstock.cl","password":"vendedor123"}'
$headersVendedor = @{ Authorization = "Bearer $($vendedor.token)" }

# 2. Consultar catálogo y categorías con token de Vendedor
$prods = Invoke-RestMethod -Uri "http://localhost/backend-api/api/productos.php?limite=5" `
    -Method GET -Headers $headersVendedor
$cats = Invoke-RestMethod -Uri "http://localhost/backend-api/api/categorias.php" `
    -Method GET -Headers $headersVendedor

# 3. Creación y validación de duplicados (Admin)
$creado = Invoke-RestMethod -Uri "http://localhost/backend-api/api/productos.php" `
    -Method POST -Headers $headersAdmin -ContentType "application/json" `
    -Body '{"nombre_producto":"Snack Test","precio_venta":750,"stock_actual":10}'
```

```
# 4. Aumento atómico de stock con transacción (Admin)
$stock = Invoke-RestMethod -Uri "http://localhost/backend-api/api/stock.php" `
    -Method POST -Headers $headersAdmin -ContentType "application/json" `
    -Body (@{ producto_id = $creado.id; cantidad = 5 } | ConvertTo-Json)

# 5. Eliminación de producto sin ventas (Admin)
$del = Invoke-RestMethod -Uri "http://localhost/backend-api/api/productos.php?id=$($creado.id)
" `
    -Method DELETE -Headers $headersAdmin
```

6. Cumplimiento de Estándares de Calidad y Seguridad

- **Seguridad de Credenciales:** Contraseñas protegidas mediante algoritmos BCRYPT nativos de PHP. Archivo config.php fuera del control de versiones Git.
- **Protección contra Inyección SQL:** Consultas 100% parametrizadas con PDO y sentencias preparadas en todos los métodos.
- **Resiliencia Transaccional:** Las operaciones de inventario en stock.php utilizan transacciones explícitas con bloqueo FOR UPDATE para evitar condiciones de carrera.
- **Integridad Referencial:** No es posible eliminar productos con historial en detalle_ventas (HTTP 409).
- **Compatibilidad Android:** targetSdk configurado en 36 con soporte de tráfico local en emulador y manejo de errores 401 que redirige automáticamente a la pantalla de Login.

Informe emitido conforme para presentación de avances y portafolio de evidencias.